

Projet Quai Antique

Présentation de Validation Pédagogique
Version Intermédiaire en Développement

Étudiante en Développement : Larisa Faessel

Contexte pédagogique : Examen étudiant — Première remise

État de l'application : **Frontend interactif & Backend Express connectés !**

Date de génération : Mai 2026

1. Présentation du projet

En tant qu'étudiante, je présente le projet Quai Antique. Il s'agit d'une application web moderne conçue pour un restaurant gastronomique haut de gamme.

L'objectif de cette première version intermédiaire est de proposer une interface utilisateur soignée pour découvrir l'établissement, et une architecture technique déjà fonctionnelle reliant les formulaires à un serveur d'API et une base de données.

Le projet est actuellement en cours de développement actif. La base technique est entièrement posée pour permettre de futures extensions rapides (authentification utilisateur complète et panneau d'administration).



2. Cahier des charges (version simplifiée)

BESOIN CLIENT :

Disposer d'un site vitrine élégant et immersif permettant de consulter les plats, la galerie du restaurant et de faire une réservation de table en direct.

FONCTIONNALITÉS INTERFACE (FRONTEND) :

- Page d'accueil : Présentation esthétique et bouton d'action d'appel à réservation.
- La Carte / Menu : Filtres interactifs par catégories (entrées, plats, desserts).
- Galerie photos : Grille adaptative pour afficher des photos en haute définition.
- Connexion / Inscription : Maquette de formulaires d'authentification.
- Formulaire de réservation : Saisie de la date, l'heure, du nombre de convives et des allergies.

FONCTIONNALITÉS LOGIQUE (BACKEND) :

- API REST RESTful : Serveur autonome pour intercepter les requêtes du frontend.
- Validation rigoureuse : Traitement et contrôle de conformité des données soumises.
- Connexion base de données : Intégration SQL complète avec gestion de pool.

BASE DE DONNÉES (POSTGRESQL) :

- Table 'reservations' : Sauvegarde pérenne des réservations de table.
- Suivi client : Stockage des informations d'identité, de contact et d'allergies.

CONTRAINTES TECHNIQUES :

- Responsive : Adaptation fluide sur mobile, tablette et ordinateur de bureau.
- Clean Architecture : Code modulaire structuré pour favoriser l'évolutivité.
- Versionnement Git : Historique de commits régulier et propre.

3. Technologies utilisées dans le projet

Pour répondre aux exigences modernes du web, j'ai sélectionné et mis en œuvre des technologies stables et standardisées. L'accent a été mis sur la mise en place d'une architecture client-serveur découplée afin de préparer au mieux la transition vers de futurs frameworks complexes.

Couche Applicative	Technologies Choisies	Rôle & Justification Technique
Frontend / Client	HTML5, CSS3, JavaScript (ES6+)	Structure sémantique, styles réactifs (CSS Grid) et requêtes asynchrones Fetch API.
Backend / API	Node.js, Express, CORS middleware	Serveur HTTP structuré en MVC, routage REST, middleware de sécurité CORS pour l'intégration.
Base de Données	PostgreSQL (Supabase Cloud), pg	Stockage relationnel, intégrité référentielle, connexion par pool robuste pour la performance.
DevOps & Outils	Git / GitHub, Docker (Nginx), VS Code	Contrôle des versions, conteneurisation pour le serveur statique local, environnement homogène.

4. État d'avancement des fonctionnalités

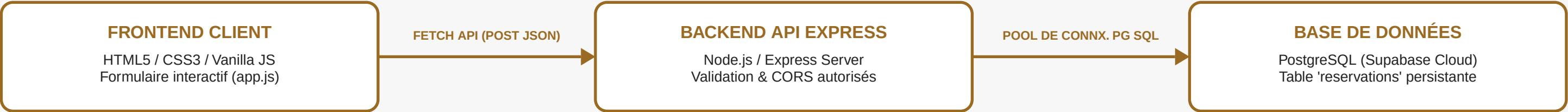
Fonctionnalité principale	Détail Technique & Implémentation	Statut actuel
Page d'accueil & Vitrine	Mise en page sémantique en HTML5 et CSS3 responsive.	' Réalisé & Validé
Galerie photos des plats	Grille réactive avec photos HD chargées localement.	' Réalisé & Validé
La carte du restaurant	Onglets interactifs dynamiques animés en JavaScript (app.js).	' Réalisé & Validé
Formulaire de réservation	Formulaire utilisateur avec validation locale des champs obligatoires.	' Réalisé & Validé
API Backend Express	Routage REST (/api/reservations), routage de santé (/api/health).	' Réalisé & Validé
Connexion PostgreSQL	Base de données hébergée et script de création automatique de table.	' Réalisé & Validé
Liaison Formulaire !' DB	Envoi des données POST JSON du Frontend vers le Backend Express.	' Intégré & Fonctionnel
Comptes & Espace Admin	Maquette existante. Sécurisation bcrypt, authentification JWT et panel admin.	% ³ Étape 2 (En cours)

5. Architecture technique de l'application

L'application implémente une architecture découpée en couches de type Client-Serveur. La communication entre le Frontend et le Backend est asynchrone et repose sur le format standard JSON.

Fonctionnement détaillé de l'intégration :

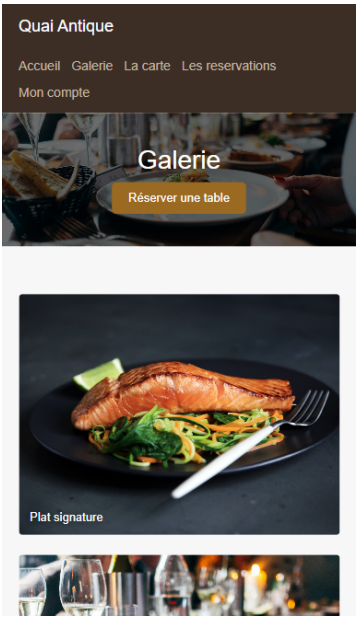
- 1. L'utilisateur remplit le formulaire de réservation sur l'interface.
- 2. Le script JavaScript intercepte la soumission, valide les types de données et effectue une requête Fetch API en POST JSON.
- 3. Le serveur Express intercepte la requête, applique les validations côté serveur (ex: nombre de convives > 0) et transmet les données au repository.
- 4. Le repository exécute une requête préparée via un pool de connexions (pg) vers PostgreSQL (Supabase) pour y insérer les informations.



6. Captures d'écran : Interface Frontend



Page d'accueil responsive



Grille de la Galerie photos



Formulaire de réservation interactif

L'interface utilisateur a été conçue pour refléter le standing d'un restaurant gastronomique. Les couleurs et polices ont été choisies avec soin (combinaison de tons sombres et dorés). Le site s'adapte de manière totalement automatique sur smartphone, tablette et écran large.

6. Architecture de code : Contrôleur Backend & DB

STRUCTURE DU CODE :

Afin de garantir un code maintenable et réutilisable, le CMC Express est organisé selon le patron Repository :

- controllers/reservation.controller.js :

Gère le traitement de la requête HTTP, extrait les paramètres de réservation, valide que tous les champs obligatoires sont bien remplis, applique des règles métier (ex: vérifier que le nombre de personnes est bien un entier strictement positif) et renvoie la réponse correspondante (201 Created ou codes d'erreur 400, 500, 503).

- repositories/reservation.repository.js :

Se charge exclusivement des interactions SQL avec PostgreSQL via des requêtes paramétrées sécurisées prévenant les injections SQL.

- config/database.js & initDatabase.js :

Gère la connexion sécurisée par pool pg et crée automatiquement la table 'reservations' au lancement du serveur.

```
// Extrait de controllers/reservation.controller.js
async function createReservation(req, res) {
  const { firstName, lastName, email, date, time, guests, allergies } = req.body;

  if (!firstName || !lastName || !email || !date || !time || !guests) {
    return res.status(400).json({
      message: 'Tous les champs obligatoires doivent etre remplis'
    });
  }

  const guestCount = Number(guests);
  if (!Number.isInteger(guestCount) || guestCount <= 0) {
    return res.status(400).json({
      message: 'Le nombre de personnes doit etre un nombre entier positif'
    });
  }

  try {
    const reservation = await reservationRepository.createReservation({
      firstName, lastName, email, date, time, guests: guestCount, allergies
    });
    return res.status(201).json({ message: 'Reservation enregistree', reservation });
  } catch (error) {
    // Gestion gracieuse d'une base de donnees indisponible
    if (error.message.includes('DATABASE_URL')) {
      return res.status(503).json({ message: 'Base de donnees indisponible' });
    }
    return res.status(500).json({ message: 'Erreur serveur' });
  }
}
```


7. Gestion des versions : Git & GitHub

MÉTHODOLOGIE DE TRAVAIL :

Le suivi des versions est un pilier de mon apprentissage. Dès le début du projet, j'ai mis en place un dépôt Git local et je l'ai connecté à mon compte GitHub afin de sauvegarder mon travail et de le rendre disponible pour la validation pédagogique.

- Organisation des commits :

Chaque commit correspond à une tâche claire et unitaire de développement. Les messages de commit respectent les conventions afin de faciliter la lecture de l'historique.

- Sécurisation du code :

J'ai veillé à mettre en place un fichier `.gitignore` robuste pour empêcher la fuite de secrets ou de dépendances lourdes (exclusion de `'node_modules'`, des fichiers `'.env'` de configuration locale contenant les identifiants de la base de données, et des journaux d'erreurs).

LIEN DU DÉPÔT GITHUB DU PROJET :

https://github.com/catcodecat/Quai_Antique_Project

LIEN DU SITE DÉPLOYÉ (FRONTEND VITRINE) :

<https://quai-antique-projet.netlify.app/>



```
$ git log -n 5
```

```
commit a3db5b7b29722234947b3e859ae58e804eeaece9
Author: Larisa Faessel <faessel.larisa@gmail.com>
Date:   Wed May 20 15:10:59 2026 +0200
```

```
    feat: connect reservation form to backend and database
```

```
commit 3775f613cc6e69c3ae4624fe19056fc048a6c830
Author: Larisa Faessel <faessel.larisa@gmail.com>
Date:   Wed May 20 11:37:04 2026 +0200
```

```
    Test backend API routes
```

```
commit d5656dc31774aa6f1d0533e7239537890df40004
Author: Larisa Faessel <faessel.larisa@gmail.com>
Date:   Tue May 19 17:46:23 2026 +0200
```

```
    fix backend startup
```

```
commit 92ce0119a780e97e445d92b94d40763bb3f32ae7
Author: Larisa Faessel <faessel.larisa@gmail.com>
Date:   Tue May 19 17:24:18 2026 +0200
```

```
    add backed Node.js Express
```

8. Difficultés rencontrées et solutions

En tant qu'étudiante en développement web, ce projet Quai Antique a été une formidable opportunité d'apprentissage mais aussi un vrai défi technique. Voici les principales difficultés que j'ai rencontrées et comment j'ai réussi à les surmonter :

- La gestion de la politique de sécurité CORS :

Lors des premiers tests de soumission du formulaire de réservation depuis mon frontend statique, les navigateurs bloquaient les requêtes à cause des règles de sécurité CORS. J'ai surmonté cela en configurant le middleware 'cors' côté Express pour autoriser spécifiquement mon origine locale de développement.

- L'intégration et la résilience de la base de données :

Établir la connexion avec PostgreSQL (Supabase) a nécessité de bien appréhender l'asynchronisme de JavaScript. Pour éviter que le serveur Express ne plante si la base de données met du temps à répondre ou à s'éveiller au démarrage, j'ai implémenté un système de reconnexion automatique (retries) dans mon script d'initialisation de base de données.

- La validation rigoureuse des données :

Assurer la cohérence des réservations (comme vérifier que le nombre de personnes est bien un nombre entier strictement positif) m'a obligée à mettre en place une double couche de validation : dans le formulaire frontend et dans le contrôleur backend Express, pour garantir la qualité de la donnée stockée.

9. Prochaines étapes du développement

1. SÉCURISATION ET AUTHENTIFICATION DES COMPTES CLIENTS :

- Implémenter le hachage sécurisé des mots de passe dans la base de données avec la bibliothèque 'bcrypt' côté CMC5CÔ4 Express.
- Générer et valider des jetons sécurisés JWT (JSON Web Tokens) lors de la connexion pour maintenir la session de l'utilisateur.
- Protéger les routes sensibles de l'API REST grâce à des middlewares d'authentification personnalisés.

2. DÉVELOPPEMENT DU PANNEAU D'ADMINISTRATION COMPLET :

- Créer des écrans d'administration permettant de modifier dynamiquement les plats de la carte, les prix et les descriptions.
- Gérer les jours et horaires d'ouverture du restaurant de manière interactive.
- Pouvoir consulter, filtrer et valider/annuler les demandes de réservations des clients.

3. REFACTORISATION ET MIGRATION COMPOSANTS REACT + VITE :

- Refactoriser le frontend statique HTML/CSS/JS actuel dans un projet moderne React + Vite pour un développement plus fluide.
- Découper l'interface utilisateur en composants autonomes et réutilisables (ex: Header, Footer, MenuTabs, ReservationForm).
- Utiliser React Router pour gérer de manière transparente et performante la navigation entre les pages sans rechargement.

4. DÉPLOIEMENT FINAL DE L'APPLICATION EN PRODUCTION :

- Déployer la base de données finale de production PostgreSQL sur Supabase.
- Héberger l'API REST Express sur une plateforme de cloud (Render ou Fly.io) avec des variables d'environnement chiffrées.
- Publier le frontend React sur Netlify avec des redirections configurées pour les requêtes API.

Conclusion de l'étudiante

Cette première phase de développement du projet Quai Antique a été extrêmement enrichissante.

Partie d'une idée de site vitrine statique, j'ai réussi à mettre en œuvre une véritable architecture applicative web. L'interface utilisateur est entièrement responsive et interactive. Le CMC4 Express est opérationnel, gère correctement les validations côté serveur et communique de manière fluide avec la base de données PostgreSQL.

La base technique est solide, saine et robuste. Le projet est parfaitement prêt pour cette première validation pédagogique intermédiaire, et me permettra d'aborder sereinement les prochaines étapes de sécurisation et de migration React.

Je remercie mon professeur pour sa lecture et ses précieux conseils pédagogiques tout au long de mon projet d'examen !

Larisa Faessel

Étudiante en Développement Web